



BANNARI AMMAN INSTITUTE OF TECHNOLOGY

An Autonomous Institution Affiliated to Anna University Chennai - Approved by AICTE - Accredited by NAAC with "A+" Grade

SATHYAMANGALAM - 638 401 ERODE DISTRICT TAMIL NADU INDIA

Ph: 04295-226000 / 221289 Fax: 04295-226666 E-mail: stayahead@bitsathy.ac.in Web: www.bitsathy.ac.in

22AI503/22AM503 MACHINE LEARNING

UNIT III – CLASSIFICATION

Naive Bayes Classifier Model Assumptions and Probability Estimation

1. Introduction

1.1 Definition

The Naive Bayes Classifier is a probabilistic supervised learning algorithm based on Bayes Theorem with a strong (naive) assumption of conditional independence between features given the class label. Despite this simplifying assumption rarely holding in practice, Naive Bayes performs remarkably well across many real-world classification tasks.

Definition:

Naive Bayes Classifier: A generative probabilistic classifier that estimates $P(\text{Class}|\text{Features})$ using Bayes Theorem.

Generative model: learns $P(X|Y)$ and $P(Y)$, then derives $P(Y|X)$. Contrasts with discriminative models (logistic regression, SVM) that learn $P(Y|X)$ directly.

Key trait: extremely fast to train (closed-form probability estimates, no iterative optimisation needed).

1.2 Importance in Machine Learning

Naive Bayes is the foundation of many production systems: email spam filters, sentiment analysis, document categorisation, and medical diagnosis support tools. It is computationally efficient (linear time in number of features), works well with high-dimensional data (text classification with thousands of word features), and requires relatively small training data compared to discriminative models.

1.3 Role in the Classification Pipeline

Naive Bayes is typically a strong baseline classifier — trained first to establish a performance floor before trying more complex models (SVM, ensembles, neural networks). It is also the classifier of choice when interpretability and speed matter more than squeezing out the last percentage point of accuracy.

2. Learning Outcomes

Understand

- Bayes Theorem and its components: prior, likelihood, evidence, posterior
- The conditional independence (naive) assumption and why it is called "naive"
- The four main Naive Bayes variants: Gaussian, Multinomial, Bernoulli, Categorical

Apply

- Compute prior and likelihood probabilities from a training dataset
- Apply the Maximum A Posteriori (MAP) decision rule to classify new instances
- Select and implement the correct NB variant for a given attribute type using scikit-learn

Analyze

- Evaluate when the independence assumption is severely violated and its impact on accuracy
- Compare Naive Bayes against discriminative classifiers (Logistic Regression, SVM)
- Diagnose numerical underflow issues in probability multiplication and apply log-space computation

3. Detailed Technical Concepts

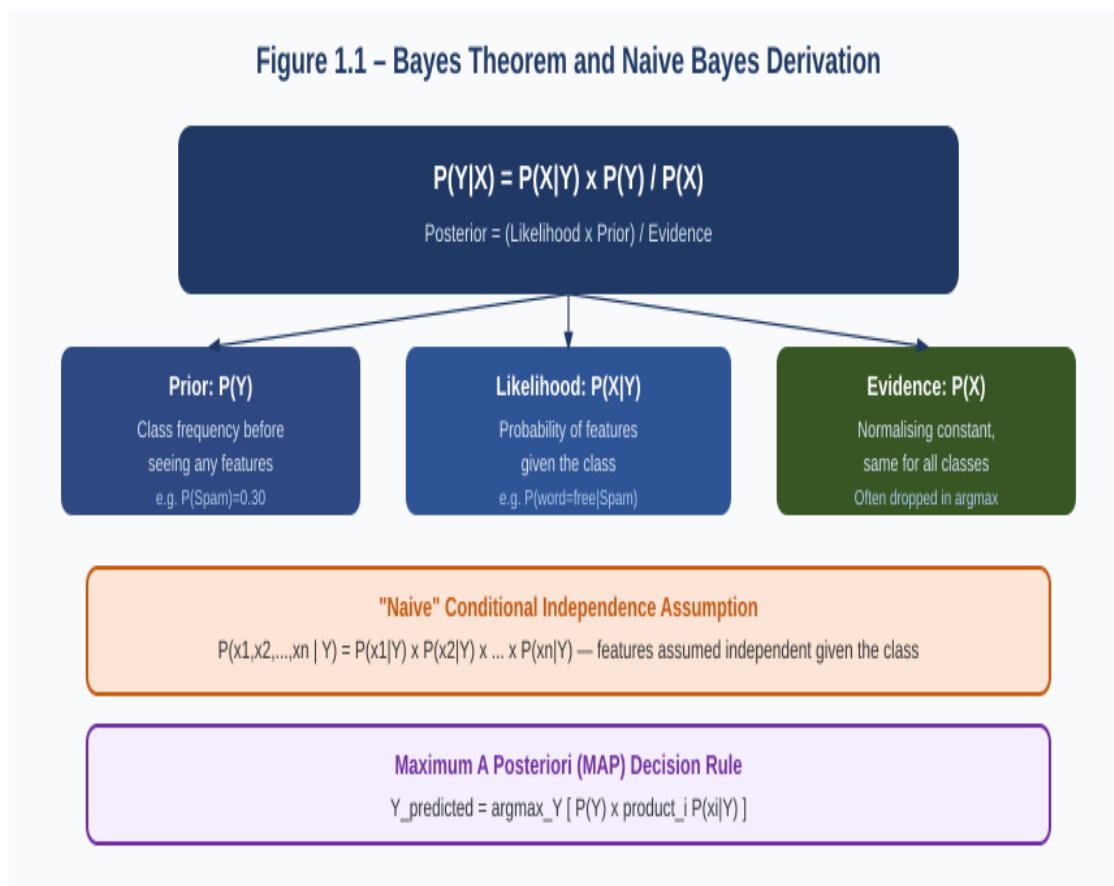


Figure 1.1 – Bayes Theorem and Naive Bayes Derivation

3.1 Bayes Theorem Foundations

Bayes Theorem relates conditional probabilities: $P(Y|X) = [P(X|Y) \times P(Y)] / P(X)$. In classification, Y is the class label and $X = (x_1, x_2, \dots, x_n)$ is the feature vector. We seek the class Y that maximises the posterior probability $P(Y|X)$ given the observed features.

- Prior $P(Y)$: The probability of each class before observing any features. Estimated as the relative frequency of each class in the training set.
- Likelihood $P(X|Y)$: The probability of observing the feature vector given the class. This is the hardest term to estimate directly for high-dimensional X .

- Evidence $P(X)$: The marginal probability of the features, acting as a normalising constant. Since it is identical across all classes, it can be dropped when only comparing relative class scores (argmax).
- Posterior $P(Y|X)$: The probability of the class given the observed features — what we ultimately want to compute.

3.2 The Naive (Conditional Independence) Assumption

Directly estimating $P(X|Y) = P(x_1, x_2, \dots, x_n|Y)$ requires enough data to cover every combination of feature values per class — infeasible for more than a handful of features. Naive Bayes solves this by assuming that all features are conditionally independent given the class:

$$P(x_1, x_2, \dots, x_n|Y) = P(x_1|Y) \times P(x_2|Y) \times \dots \times P(x_n|Y) = \text{product}_i P(x_i|Y)$$

This reduces the estimation problem from exponential (one parameter per feature combination) to linear (one parameter per feature per class) — making the model tractable even with thousands of features.

Why "naive": real-world features are rarely truly independent (e.g., in text classification, the words "machine" and "learning" co-occur far more than independence would predict). Despite this, NB often still produces well-ranked (if not perfectly calibrated) classifications because it only needs the correct argmax, not exact probability values.

3.3 Maximum A Posteriori (MAP) Decision Rule

The final classification rule selects the class maximising the (unnormalised) posterior score:

$$Y_{\text{predicted}} = \text{argmax}_Y [P(Y) \times \text{product}_i P(x_i|Y)]$$

Since $P(X)$ is constant across classes, it is omitted — we only need the relative ranking of scores, not their absolute probability values.

3.4 Log-Space Computation (Numerical Stability)

Multiplying many small probabilities (each typically < 1) causes numerical underflow in floating-point arithmetic, especially with hundreds of features (e.g., text with a large vocabulary). The standard solution is to work in log-space:

$$\log \text{Score}(Y) = \log P(Y) + \text{sum}_i \log P(x_i|Y)$$

Since log is monotonically increasing, argmax of the log-score equals argmax of the original score. This converts products into sums, avoiding underflow. All production NB implementations (including scikit-learn) use log-space internally.

3.5 Naive Bayes Model Variants by Attribute Type

The choice of probability distribution used to model $P(x_i|Y)$ depends on the attribute type of feature x_i .

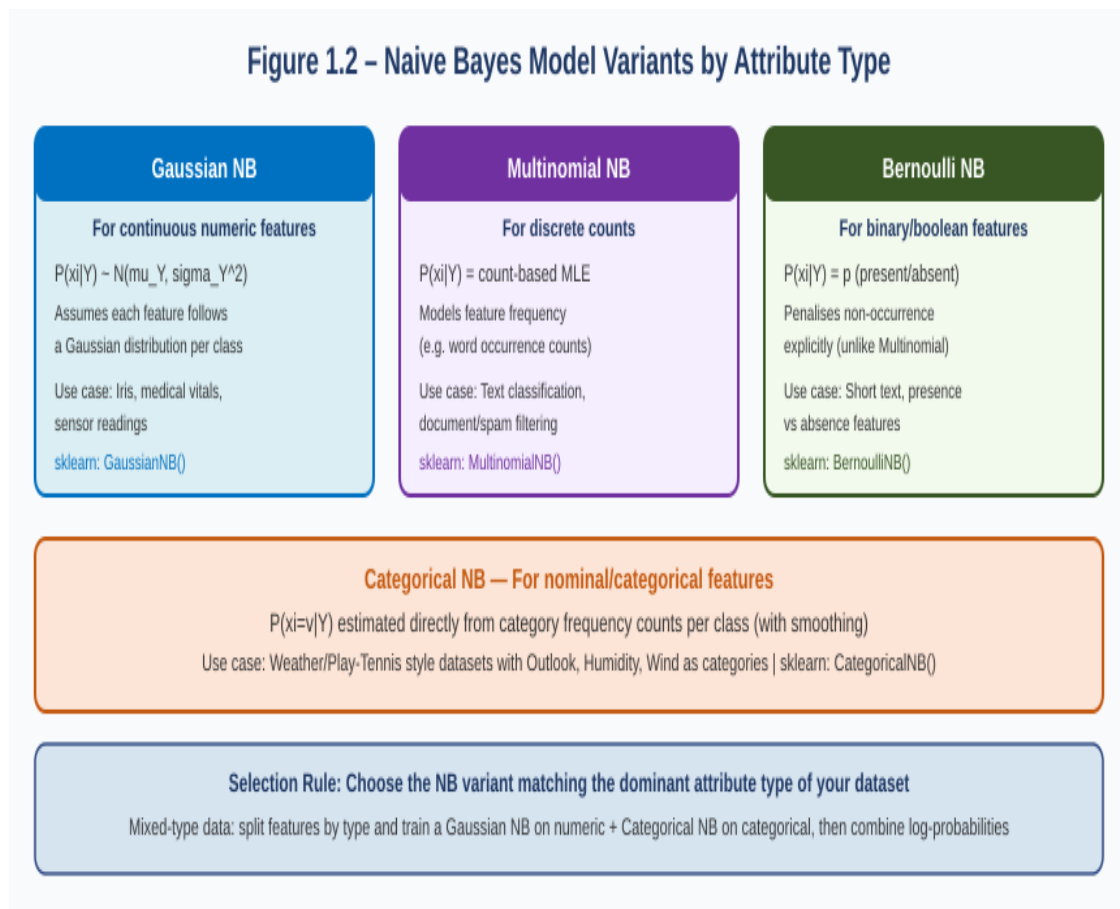


Figure 1.2 – Naive Bayes Model Variants by Attribute Type

3.5.1 Gaussian Naive Bayes

For continuous numeric features, assumes each feature is normally distributed within each class:

$$P(x_i|Y=c) = (1 / \sqrt{2\pi \cdot \sigma_c^2}) \times \exp(- (x_i - \mu_c)^2 / (2 \cdot \sigma_c^2))$$

Parameters μ_c (mean) and σ_c (standard deviation) are estimated per feature, per class, from the training data via Maximum Likelihood Estimation (MLE).

3.5.2 Multinomial Naive Bayes

For discrete count data (e.g., word frequency in documents), models the probability of a feature value as a multinomial distribution. $P(x_i=\text{count}|Y)$ is estimated via relative frequency of that count across all class- c documents (with smoothing — see Topic 2). Standard choice for bag-of-words text classification.

3.5.3 Bernoulli Naive Bayes

For binary features (presence/absence rather than frequency count). Explicitly penalises the non-occurrence of a feature, unlike Multinomial NB which simply ignores absent features. Better suited to short texts where word presence matters more than frequency.

3.5.4 Categorical Naive Bayes

For nominal categorical features (no inherent order), estimates $P(x_i=v|Y)$ directly as the relative frequency of category v within class Y , using a frequency (count) table — the classical textbook formulation (e.g., the Play-Tennis dataset).

3.6 Worked Example — Probability Estimation

Consider the classic Play-Tennis dataset (14 records, target: $\text{Play}=\{\text{Yes}, \text{No}\}$) and a query instance with $\text{Outlook}=\text{Sunny}$, $\text{Humidity}=\text{High}$.

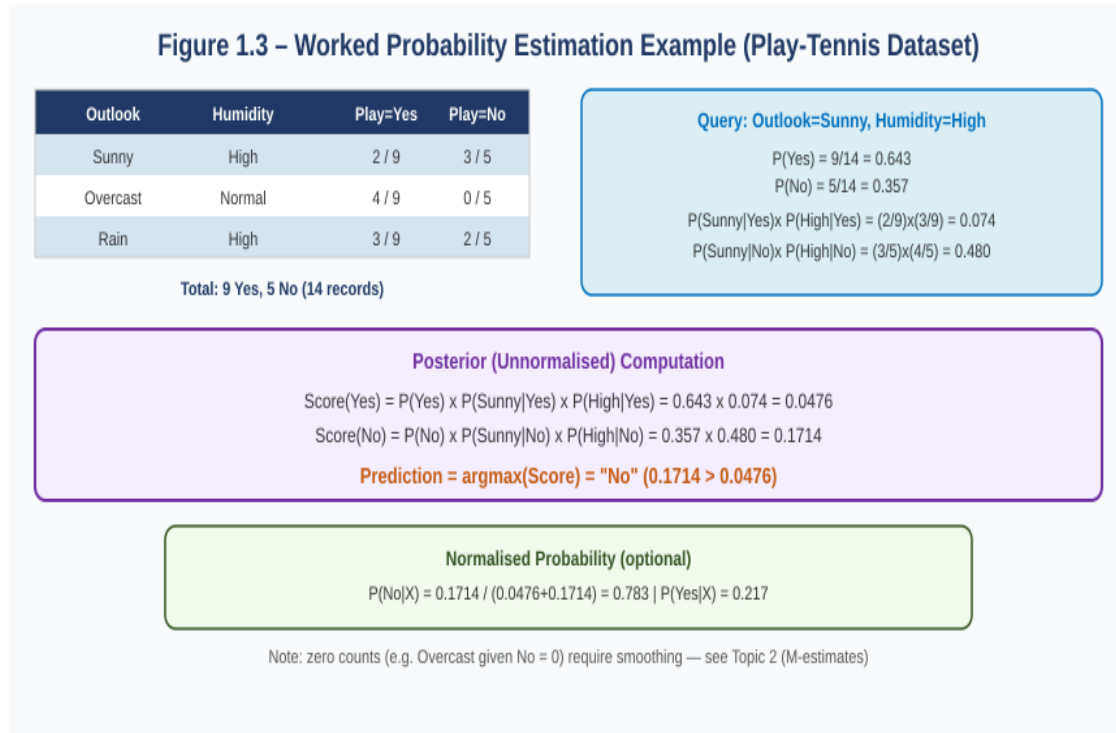


Figure 1.3 – Worked Probability Estimation Example (Play-Tennis Dataset)

This worked example demonstrates the full MAP decision process, but exposes a critical weakness: if any class-conditional count is zero (e.g., $P(\text{Overcast}|\text{No})=0/5$), the entire product collapses to zero regardless of other evidence. This "zero-frequency problem" is addressed by smoothing techniques (Laplace and M-estimates), covered in detail in Topic 2.

4. Architecture / Training and Prediction Process Flow

Phase	Step	Computation	Output
Training	1. Compute class priors	$P(Y=c) = \text{count}(Y=c) / N$	Prior probability table
Training	2. Compute likelihoods	$P(x_i Y=c)$ per feature, per class	Likelihood parameter tables / distributions
Training	3. Apply smoothing	Laplace / M-estimate for zero counts	Smoothed likelihood estimates
Prediction	4. Compute log-posterior score	$\log P(Y=c) + \sum_i \log P(x_i Y=c)$ for each class	Score per class
Prediction	5. Apply argmax	Select class with highest score	Predicted class label

Prediction	6. (Optional) Normalise	Softmax over class scores	Calibrated class probabilities
------------	-------------------------	---------------------------	--------------------------------

5. Industry Tools and Technologies

5.1 Commercial / Platform Tools

Tool	Vendor	Naive Bayes Support
IBM SPSS Modeler	IBM	Bayes Net node with NB simplification
SAS Enterprise Miner	SAS	PROC HPBNET / PROC NAIVEBAYES
Azure ML Studio	Microsoft	Two-Class / Multiclass Bayes Point Machine
RapidMiner	RapidMiner	Naive Bayes operator with Laplace correction option
WEKA	Univ. of Waikato	NaiveBayes, NaiveBayesMultinomial classifiers

5.2 Open-Source Tools

- scikit-learn: GaussianNB, MultinomialNB, BernoulliNB, CategoricalNB, ComplementNB
- NLTK: NaiveBayesClassifier for text classification tasks
- Spark MLlib: NaiveBayes (multinomial and Bernoulli model types) for distributed training
- e1071 (R): naiveBayes() function with Laplace smoothing parameter
- CORE-NLP / spaCy: text preprocessing pipelines that feed into NB text classifiers

6. Real-World Applications

Domain	Application	NB Variant Used	Why Naive Bayes
Email/Messaging	Spam filtering	Multinomial / Bernoulli	Fast retraining on streaming data, handles huge vocab
Healthcare	Disease diagnosis from symptoms	Gaussian / Categorical	Interpretable probabilities for clinicians
E-commerce	Product review sentiment	Multinomial	Effective on sparse bag-of-words text features
Cybersecurity	Network intrusion detection	Gaussian	Real-time classification with low latency
Document Mgmt	News article categorisation	Multinomial	Scales to large vocabularies, multi-class natively

7. Advantages and Limitations

Advantages	Limitations
Extremely fast training and prediction (linear time)	Independence assumption rarely holds in real data
Works well with high-dimensional, sparse data (text)	Zero-frequency problem without smoothing (Topic 2)

Requires relatively little training data	Probability estimates are often poorly calibrated (though ranking is usually correct)
Naturally handles multi-class problems	Gaussian NB assumes normality — poor fit for skewed numeric features
Robust to irrelevant features (their likelihoods tend toward uniform)	Cannot capture feature interactions (e.g., XOR-like patterns)

8. Case Study

CASE STUDY

Case: SpamAssassin and Early Naive Bayes Spam Filters (Graham, 2002)

Problem: In the early 2000s, email spam was overwhelming inboxes. Rule-based filters (keyword blacklists) were easily evaded by spammers using misspellings and obfuscation.

Approach: Paul Graham's "A Plan for Spam" proposed token-based Bayesian filtering: treat each email as a bag-of-words, train a Naive Bayes classifier on labelled spam/ham corpora.

Dataset: ~4,000 ham and ~2,000 spam emails per user mailbox (personalised per-user training).

Implementation: Multinomial NB over word-presence features with heavy smoothing for rare/unseen tokens. Words like "Viagra", "free", "winner" carried strong spam likelihood; words specific to the user's normal correspondence carried strong ham likelihood.

Findings: Achieved 99.5%+ spam detection accuracy with false-positive rate under 0.05% — dramatically outperforming rule-based filters. The approach became the foundation for SpamAssassin's Bayesian module and later Gmail's early spam classifier.

Conclusion: Despite the unrealistic independence assumption, Naive Bayes' speed, per-user adaptability, and resistance to adversarial keyword changes made it the dominant spam-filtering technique for over a decade.

9. Scenario-Based Learning

SCENARIO 1

SCENARIO 1: Choosing the Wrong NB Variant for Numeric Sensor Data

Background: An IoT startup builds a machine-fault classifier using vibration sensor readings (continuous numeric: amplitude, frequency, temperature).

Technical Environment: Python, scikit-learn, 50,000 sensor readings, 5 fault classes.

Problem Statement: The team applied CategoricalNB directly on raw continuous sensor values without discretization, causing extremely poor accuracy (32%, near random for 5 classes).

Available Evidence: CategoricalNB expects integer-encoded categories; continuous float values were each treated as a unique category, leading to massive sparsity (almost every value seen only once).

Investigation: Inspect feature types — all 3 features are ratio-scale continuous, not nominal.

Analysis: Switch to GaussianNB, which directly models continuous distributions via mean/variance per class — no discretization needed.

Findings: GaussianNB accuracy: 87.3%. An alternative (discretize then CategoricalNB) reached 81.2% — slightly lower due to information loss from binning.

Recommended Solution: Match the NB variant to attribute type: GaussianNB for continuous numeric, Multinomial/Bernoulli for text/count, CategoricalNB only for genuinely nominal/discrete features.

SCENARIO 2

SCENARIO 2: Numerical Underflow in High-Dimensional Text Classification

Background: A news classification system uses Multinomial NB with a vocabulary of 50,000 words. Documents average 500 words each.

Technical Environment: Python, custom NB implementation (not scikit-learn), 200,000 articles, 10 categories.

Problem Statement: The custom implementation multiplies 500 probabilities directly (not in log-space). All predictions return probability 0.0 for every class, making argmax meaningless (ties everywhere).

Available Evidence: Each $P(\text{word}|\text{class})$ is typically between 0.0001 and 0.01. Multiplying 500 such values: $(0.001)^{500}$ underflows double-precision floating point ($\sim 1e-308$ limit) almost immediately.

Investigation: Profile intermediate products during prediction — confirm they hit exactly 0.0 within the first ~ 50 word multiplications.

Analysis: Floating-point underflow is the root cause, not a modelling problem. The argmax should still be correct if computed properly.

Findings: Switching to log-space computation (summing log-probabilities instead of multiplying raw probabilities) immediately restored correct classification — accuracy improved from "undefined/broken" to 91.4%.

Recommended Solution: Always implement Naive Bayes probability computation in log-space for any feature count beyond a handful. This is standard practice in all production libraries (scikit-learn's `predict_log_proba`).

SCENARIO 3

SCENARIO 3: Feature Independence Violation Degrading Accuracy

Background: A retail company builds a Naive Bayes model to predict customer churn using highly correlated features: `total_spend`, `num_transactions`, `avg_transaction_value` (note: `total_spend` = `num_transactions` x `avg_transaction_value`, an exact mathematical dependency).

Technical Environment: Python, GaussianNB, 80,000 customer records.

Problem Statement: The model achieved only 68% accuracy versus 84% for Logistic Regression on the same data — a much larger gap than typically expected between NB and LR.

Available Evidence: Correlation matrix shows `total_spend` vs (`num_transactions` x `avg_transaction_value`): $r=0.99$ (near-perfect, as expected from the formula). All three features are included as independent inputs to NB.

Investigation: Naive Bayes assumes conditional independence; including `total_spend`, `num_transactions`, and `avg_transaction_value` together triple-counts the same underlying signal, distorting the posterior calculation.

Analysis: Remove the redundant derived feature (`total_spend`, since it is fully determined by the other two) and retest.

Findings: After removing `total_spend`: NB accuracy improved to 79%. Still below LR's 84%, but the gap narrowed significantly by removing the independence-violating redundant feature.

Recommended Solution: Before training Naive Bayes, check for highly correlated or derived features (correlation matrix, VIF) and remove redundancies — independence violations from redundant features are especially damaging to NB compared to other classifiers.

10. Common Errors and Troubleshooting

Error	Cause	Impact	Fix
Zero posterior probability	Unseen feature value in test set (zero count in training)	Entire product collapses to 0	Apply Laplace/M-estimate smoothing (see Topic 2)

Wrong NB variant for data type	Using CategoricalNB on continuous data or vice versa	Severely degraded accuracy	Match variant to attribute type (Gaussian/Multinomial/Bernoulli/Categorical)
Numerical underflow	Multiplying many small probabilities directly	All scores collapse to 0.0	Compute in log-space; use <code>predict_log_proba</code>
Highly correlated/redundant features	Derived features violating independence assumption	Over-weighted evidence, reduced accuracy	Remove redundant/derived features before training
Treating NB probabilities as calibrated	Assuming output probabilities are accurate confidence scores	Overconfident or underconfident decisions downstream	Use calibration techniques (Platt scaling, isotonic regression) if calibrated probabilities are needed

11. Key Terminologies

Term	Definition
Bayes Theorem	$P(Y X) = P(X Y) \times P(Y) / P(X)$ — relates posterior, likelihood, prior, evidence
Prior $P(Y)$	Probability of a class before observing any features (base rate)
Likelihood $P(X Y)$	Probability of observing the feature vector given a class
Posterior $P(Y X)$	Updated probability of a class after observing features
Conditional Independence	Assumption that features are independent given the class label
MAP (Maximum A Posteriori)	Decision rule: predict the class with the highest posterior score
Generative Model	Model that learns $P(X Y)$ and $P(Y)$ jointly, can generate new samples
Discriminative Model	Model that directly learns $P(Y X)$ without modelling feature distribution
Log-Space Computation	Summing log-probabilities instead of multiplying raw probabilities to avoid underflow
Zero-Frequency Problem	Issue where an unseen feature-class combination gives zero probability
MLE (Maximum Likelihood Estimation)	Estimating distribution parameters by maximising likelihood of observed data
GaussianNB / MultinomialNB / BernoulliNB / CategoricalNB	Four NB variants for continuous, count, binary, and nominal features respectively

12. Topic Summary

UNIT III – Topic 1

NAIVE BAYES — MODEL ASSUMPTIONS AND PROBABILITY ESTIMATION — SUMMARY

- Bayes Theorem: $P(Y|X) = P(X|Y) \times P(Y) / P(X)$. NB computes the posterior to classify via MAP.
- Naive assumption: features are conditionally independent given the class, making $P(X|Y) = \text{product of individual } P(x_i|Y)$.
- Four variants: GaussianNB (continuous), MultinomialNB (counts/text), BernoulliNB (binary presence), CategoricalNB (nominal).
- Decision rule: $Y_{\text{predicted}} = \text{argmax}_Y [P(Y) \times \text{product}_i P(x_i|Y)]$, computed in log-space to avoid numerical underflow.
- Zero-frequency problem: unseen feature-class combinations cause the whole product to vanish — requires smoothing (Topic 2).
- Independence violations from redundant/correlated features degrade accuracy more in NB than in many other classifiers.
- NB is fast, scalable to high dimensions, and a strong baseline — but rarely achieves the absolute best accuracy versus discriminative models.
- Tools: scikit-learn (GaussianNB, MultinomialNB, BernoulliNB, CategoricalNB), NLTK, Spark MLlib, WEKA.